

Grab'N'Go: In-Depth Code & Architecture Review

This document provides a detailed, line-by-line and block-by-block explanation of the core files in the Grab'N'Go rental platform. It is designed to help you explain exactly how the code works during a project presentation or review.

Part 1: Backend Architecture (Node.js + Express + PostgreSQL)

The backend acts as the brain of the application. It connects to the database, enforces security (authentication/authorization), and provides data to the frontend via a RESTful API.

1. The Entry Point: backend/src/server.js

This file starts the entire backend server.

```
require('dotenv').config(); // Loads secret variables from a .env file into process.env
const express = require('express'); // Imports the Express backend framework
const cors = require('cors'); // Imports CORS middleware to allow the frontend to talk to this backend
const path = require('path'); // Utility for working with file paths
const db = require('./config/db'); // Imports our custom database connection pool

const app = express(); // Creates the Express application instance
const PORT = process.env.PORT || 5000; // Define the port the server will run on

// Middleware Setup
app.use(cors()); // Enables Cross-Origin Resource Sharing (crucial for React to call Express)
app.use(express.json()); // Tells Express to parse incoming requests with JSON payloads (e.g., req.body)
app.use(express.urlencoded({ extended: true })); // Parses URL-encoded bodies (like standard HTML forms)

// Static Files
app.use('/uploads', express.static(path.join(__dirname, '..', 'uploads')));
// Exposes the 'uploads' folder so images can be viewed via URL (e.g., http://localhost:5000/uploads/image.png)

// Database Health Check
app.get('/health', async (req, res) => {
  try {
    const result = await db.query('SELECT NOW()'); // Queries Postgres for the current time
    res.json({ status: 'ok', time: result.rows[0].now }); // Sends back a success JSON response
  } catch (err) {
    res.status(500).json({ status: 'error', message: 'Database connection failed' });
  }
});

// Route Imports
const userRoutes = require('./modules/users/user.routes');
const categoryRoutes = require('./modules/categories/category.routes');
// ... other imports ...

// Mounting Routes
app.use('/api', userRoutes); // Any request to /api/* goes to the user routes (login, register)
app.use('/api/categories', categoryRoutes); // Any request to /api/categories goes here
// ... other mount points ...

// Global Error Handler
app.use((err, req, res, next) => {
  // If any route throws an error, it gets caught here so the server doesn't crash
  console.error(err.stack);
  res.status(err.statusCode || 500).json({ message: err.message || 'Internal Server Error' });
});

// Start Server
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`); // Stars listening on port 5000
});
```

2. Database Connection: backend/src/config/db.js

This file connects the Node.js app to the PostgreSQL database.

```
const { Pool } = require('pg'); // Imports the connection Pool from the 'pg' (postgres) library

// Create a new connection pool using environment variables
const pool = new Pool({
  user: process.env.DB_USER, // Database username (e.g., postgres)
  host: process.env.DB_HOST, // Database host (e.g., localhost)
  database: process.env.DB_NAME, // Database name (e.g., rental_platform)
  password: process.env.DB_PASSWORD, // Database password
  port: process.env.DB_PORT, // Database port (usually 5432)
});

// Export the 'query' method so other files can run SQL commands easily
module.exports = {
  query: (text, params) => pool.query(text, params),
  pool // Export the raw pool in case we need transactions
};
```

3. Security: backend/src/middlewares/authMiddleware.js

This file protects routes so only logged-in users can access them.

```
const jwt = require('jsonwebtoken'); // Imports JSON Web Token library

const authenticateToken = (req, res, next) => {
  // Look for the "Authorization" header in the incoming request
  const authHeader = req.headers['authorization'];

  // The header format is "Bearer <token>". We split by space to get just the token.
  const token = authHeader && authHeader.split(' ')[1];

  // If no token exists, reject the request with 401 Unauthorized
  if (!token) {
    return res.status(401).json({ message: 'Authentication required' });
  }

  // Verify the token using our secret key
  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      // If the token is fake or expired, return 403 Forbidden
      return res.status(403).json({ message: 'Invalid or expired token' });
    }

    // If valid, attach the decrypted user data (id, role) to the req object
    req.user = user;

    // Move on to the actual controller function
    next();
  });
};

// Helper middleware to restrict access to specific roles
const requireRole = (roles) => {
  return (req, res, next) => {
    // Check if the user's role (extracted from the token) is in the allowed list
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Access denied. Insufficient permissions.' });
    }
    next();
  };
};

module.exports = { authenticateToken, requireRole };
```

Part 2: Frontend Architecture (React + Vite + Tailwind)

The frontend provides the User Interface (UI). It manages state (what the user sees) and communicates with the backend APIs.

1. The Entry Point: frontend/src/main.jsx

This is the very first file React runs.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx'; // The main app component
import './index.css'; // Global CSS (Tailwind imports)
import { ThemeProvider } from './contexts/ThemeContext'; // Our Dark/Light mode context

// Finds the <div id="root"> in index.html and injects React into it
ReactDOM.createRoot(document.getElementById('root')).render(
  // StrictMode helps find bugs during development
  <React.StrictMode>
    {/* ThemeProvider wraps the app so ALL components know if it's dark or light mode */}
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </React.StrictMode>,
);
```

2. State & Security: frontend/src/context/AuthContext.jsx

This file remembers who is logged in across the whole website.

```

import { createContext, useContext, useState, useEffect } from 'react';
import axios from 'axios'; // Tool for making HTTP requests

// Create a context (like a global variable container)
const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  // State to hold the current user's profile data
  const [user, setUser] = useState(null);
  // State to prevent flickers while checking if someone is logged in
  const [loading, setLoading] = useState(true);

  // This runs ONCE when the website is freshly reloaded
  useEffect(() => {
    const checkLoggedIn = async () => {
      // Check the browser's local memory for an existing token
      const token = localStorage.getItem('token');
      if (token) {
        try {
          // If a token exists, ask the backend "Who does this belong to?"
          const res = await axios.get('http://localhost:5000/api/profile', {
            headers: { Authorization: `Bearer ${token}` }
          });
          // Save the user data into our state
          setUser(res.data.user);
        } catch (error) {
          // If the token is expired/invalid, delete it
          localStorage.removeItem('token');
        }
      }
      setLoading(false); // Done loading
    };
    checkLoggedIn();
  }, []);

  // Login Function called by Login.jsx
  const login = async (email, password) => {
    const response = await axios.post('http://localhost:5000/api/login', { email, password });
    // The backend replies with a token and user data. Save both.
    localStorage.setItem('token', response.data.token);
    setUser(response.data.user);
    return response.data;
  };

  // Logout Function
  const logout = () => {
    localStorage.removeItem('token'); // Delete token
    setUser(null); // Clear user state
  };

  // Provide these functions and data to the rest of the app
  return (
    <AuthContext.Provider value={{ user, login, logout, register, loading }}>
      {!loading && children}
    </AuthContext.Provider>
  );
};

// Custom hook so other files can just type `const { user } = useAuth();`
export const useAuth = () => useContext(AuthContext);

```

3. API Communication: frontend/src/services/api.js

This file configures Axios to automatically attach tokens to requests.

```
import axios from 'axios';

// Create a base instance of axios pointing to our backend URL
const api = axios.create({
  baseURL: 'http://localhost:5000/api', // Backend base URL
});

// Request Interceptor: Before ANY request leaves the frontend...
api.interceptors.request.use(
  (config) => {
    // Grab the token from LocalStorage
    const token = localStorage.getItem('token');
    if (token) {
      // If it exists, attach it to the Authorization header
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

export default api;
```

4. User Interface Example: frontend/src/pages/Login.jsx

How a React page actually renders and uses the logic.

```

import { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { useNavigate, Link } from 'react-router-dom';
import ThemeToggle from '../components/ThemeToggle';

const Login = () => {
  // 1. Component State: Variables holding what the user types in the boxes
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  // 2. Hooks: Pull in functions from Context and Router
  const { login } = useAuth(); // Our login function from AuthContext
  const navigate = useNavigate(); // Tool to redirect the user to a new page

  // 3. Form Submission Handler
  const handleSubmit = async (e) => {
    e.preventDefault(); // Stops the page from refreshing when clicking submit
    try {
      // Attempt to login
      const data = await login(email, password);

      // If successful, check their role and redirect them to the correct dashboard
      if (data.user.role === 'admin') navigate('/admin/dashboard');
      else if (data.user.role === 'shop_owner') navigate('/shop-owner/dashboard');
      else navigate('/dashboard');
    } catch (err) {
      // If it fails, display the error message on screen
      setError(err.response?.data?.message || 'Login failed');
    }
  };

  // 4. JSX Rendering: The visible HTML
  return (
    // The classes here are Tailwind CSS.
    // Example: 'bg-gray-50 dark:bg-[#0B0F19]' means light gray in light mode, dark navy in dark mode.
    <div className="relative flex items-center justify-center min-h-screen bg-gray-50 dark:bg-[#0B0F19]">

      {/* The Sun/Moon Toggle Component */}
      <div className="absolute top-6 right-6">
        <ThemeToggle />
      </div>

      {/* The Login Card */}
      <div className="bg-white dark:bg-[#111827] p-10 rounded-3xl w-full max-w-md">

        {/* HTML Form calling handleSubmit when submitted */}
        <form onSubmit={handleSubmit} className="space-y-5">
          <div>
            <label>Email</label>
            {/* Input field tied to the 'email' state */}
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)} // Updates state as user types
              required
            />
          </div>
          <div>
            <label>Password</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)} // Updates state as user types
              required
            />
          </div>
          <button type="submit" className="w-full bg-emerald-600 text-white">
            Sign In
          </button>
        </form>

      </div>
    </div>
  );
};

export default Login;

```

Part 3: Database Tables & Flow

When the user clicks "Sign In", what happens in PostgreSQL?

1. The frontend hits `POST /api/login`.
2. The backend controller runs a SQL query:

```
SELECT * FROM users WHERE email = $1;
```

3. The `users` table holds columns: `id`, `email`, `password` (hashed).
4. The backend compares the hashed password using `bcrypt`. If it matches, it signs a JWT using the user's `id` and `role`.
5. The frontend saves that JWT, and every subsequent request to buy an item or view a booking checks that token to find the `id` of the buyer.